Worksheet 4 — Injection & Input Handling (3 hrs)

> **Course:** Software Security (KOSEN69) · **Week 4 Aligned:** OWASP 2025 **A05 Injection** · **CWE-89** (SQLi), **CWE-78** (OS command injection), **CWE-434** (unrestricted upload) **Signature game:** 🐍 **SQLi Boss Fight** — each successful injection lands a "hit" on the boss; the boss falls when you dump every credential and land an RCE.

> ⚠️ **Ethics note:** All payloads here are for the provided sandbox (`vulnerable_app.py`) and your own DVWA/Juice Shop containers **only**. Never test systems you do not own or have written permission to test. Unauthorized injection is a crime under most computer-misuse laws.

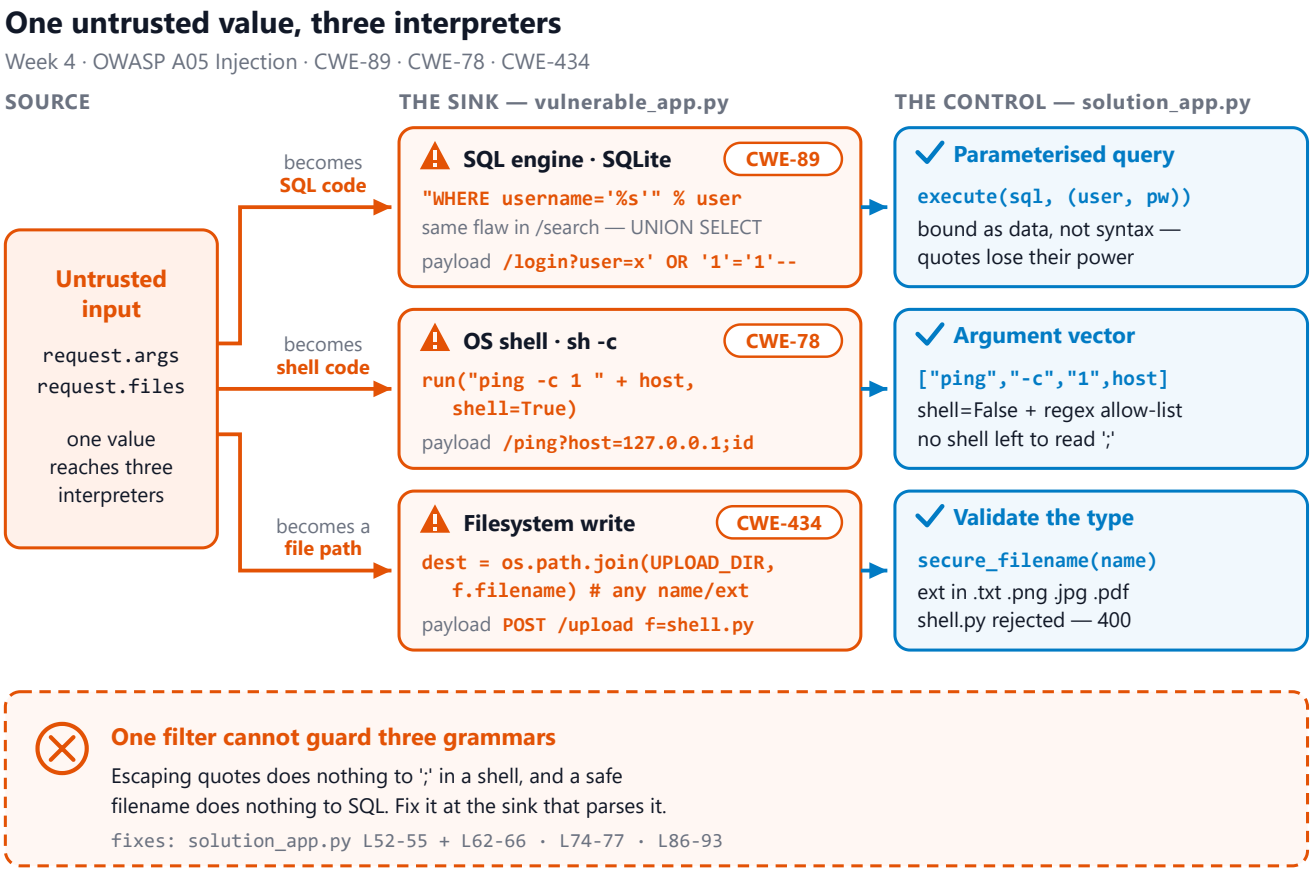
Part 1 — Student Information

Name	Student ID	Date	Group
Supichaya Rabiabnaweenurak	6631503118	13/9/2569	---

Part 2 — Lecture Questions

Answer in 2–4 sentences each.

1. Why does a **parameterized query** (`execute(sql, (params,))`) defeat SQL injection, while string formatting (`"... '%s'" % user`) does not? Reference how the database treats data vs. code.
2. In the `/ping` endpoint, `subprocess.run("ping -c 1 " + host, shell=True)` is vulnerable. Explain how `shell=True` turns user input into **CWE-78**, and how an argument array (`["ping","-c","1",host]`) removes the shell.
3. Distinguish **input validation** (allow-list) from **output handling**. Why is validation alone insufficient defense for SQLi?
4. The `/upload` route saves any filename to disk (**CWE-434**). What two properties must a directory and a filename have for an upload to become remote code execution, and which does `solution_app.py` remove?
5. What is a **UNION-based** SQLi, and why must the injected `SELECT` return the same number of columns as the original query? Relate to `/search?q=' UNION SELECT username,password FROM users--`.



Part 3 — Hands-on Lab (150 min)

Learning goals: extract data via SQLi, achieve OS command injection, exploit an unrestricted upload, then prove each fix in `solution_app.py` blocks the payload.

Prerequisites: Docker + Docker Compose, `curl`, a browser. Working dir: `labs/week04-injection/`.

Environment setup

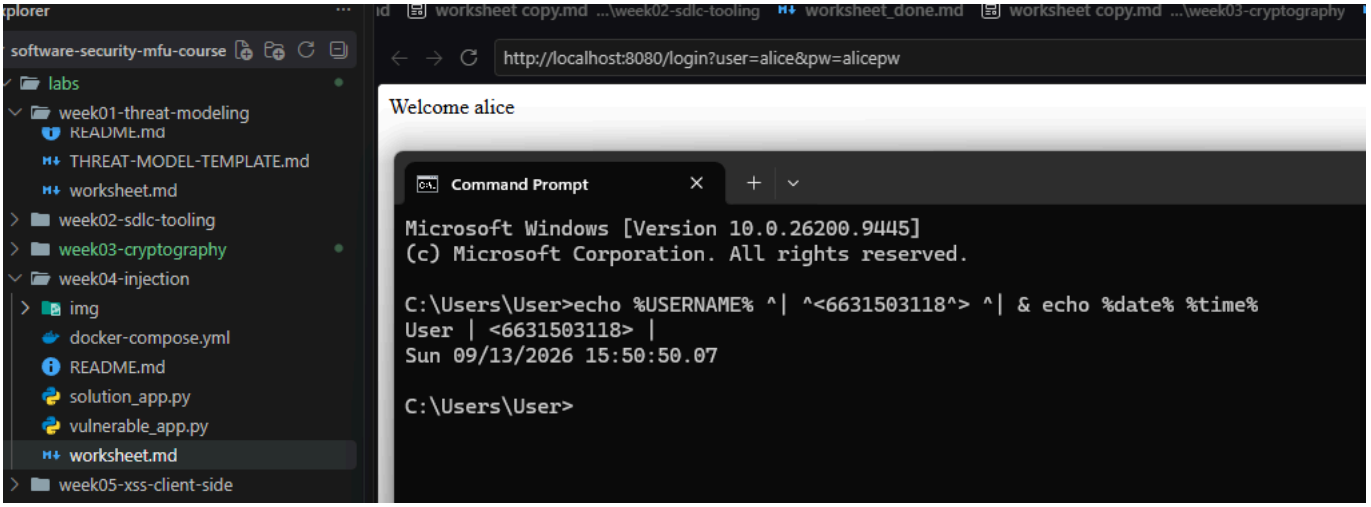
```
cd labs/week04-injection
docker compose up            # builds python:3.12-slim, installs flask, runs vulnerable_app.py
# vulnerable app -> http://localhost:8080   (service name: injection-lab, port 8080)
```

Optional secondary targets:

```
docker run --rm -it -p 80:80 vulnerables/web-dvwa        # DVWA  -> http://localhost
docker run --rm -p 3000:3000 bkimminich/juice-shop       # Juice Shop -> http://localhost:3000
```

What to submit per task: the exact **payload/command**, a **screenshot** of the response proving success, and a **2–3 sentence mitigation** in your own words.

Task 0 — Onboarding (5 min). Browse to `http://localhost:8080/login?user=alice&pw=alicepw` and confirm `Welcome alice`. Note the seeded users (`alice`, `bob`). Screenshot the working app. *Deliverable: screenshot.*

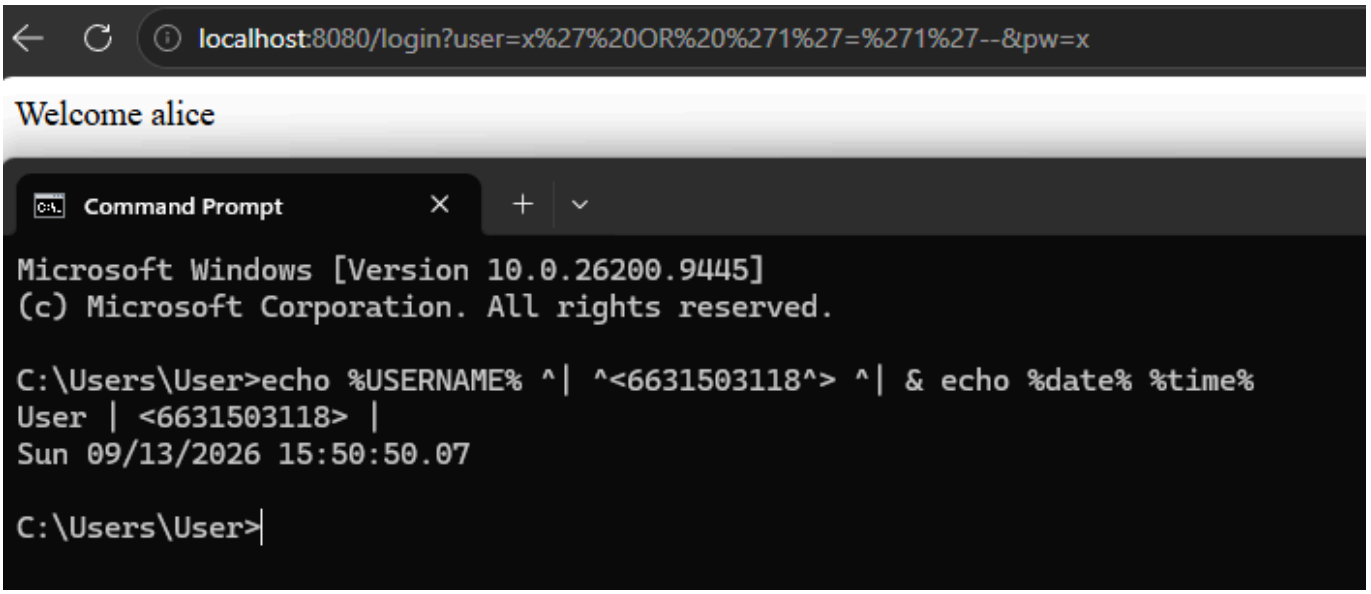


Before you start — see why concatenation is the flaw 🔬 Type any input and watch which characters the database will parse as *SQL* rather than as a name. The point is not the payload; it is that with concatenation the input becomes syntax, and with a parameterised query it structurally cannot. You will be asked to state that difference in your own words in Task 5.

```
sqli-parse
```

Task 1 — Auth bypass via SQLi (25 min) 🐍 Hit #1.

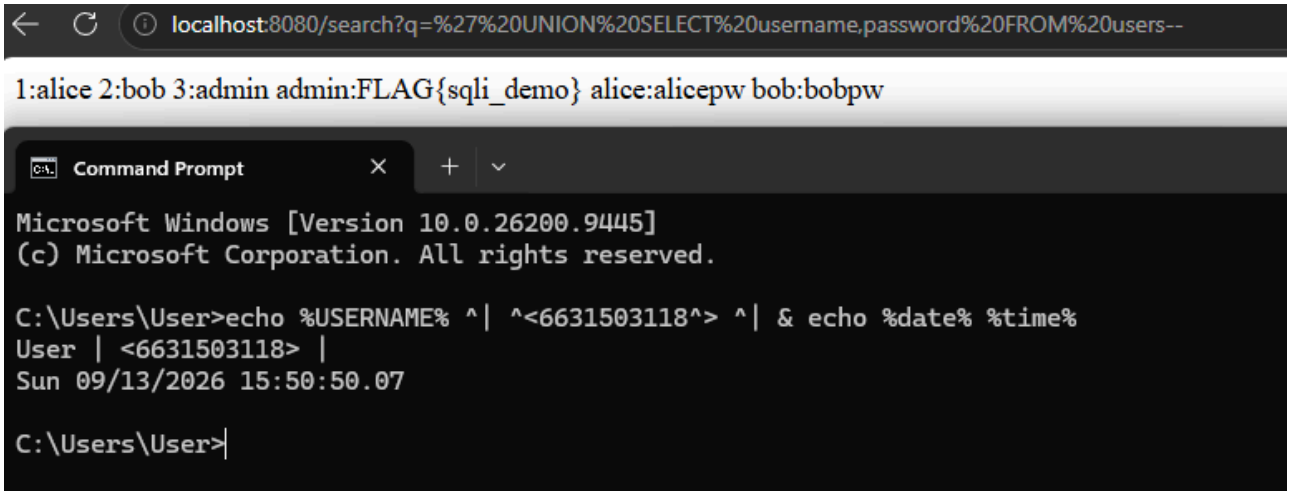
- *Goal:* log in as `alice` with **no valid password**.
- *Steps:* hit `/login?user=alice'--&pw=x`, then `/login?user=x' OR '1'='1'--&pw=x` (the trailing `--` is required: without it, SQL binds `AND` tighter than `OR`, so `... OR '1'='1' AND password='x'` matches no row). Observe the comment in the query at lines 61–63 of `vulnerable_app.py`.
- *Deliverable:* both URLs + screenshot of `Welcome alice` + explain why `--` and `OR '1'='1` work.



- the -- comments out the trailing AND password='x' so the query stops there, and OR '1'='1' makes the WHERE clause always true.

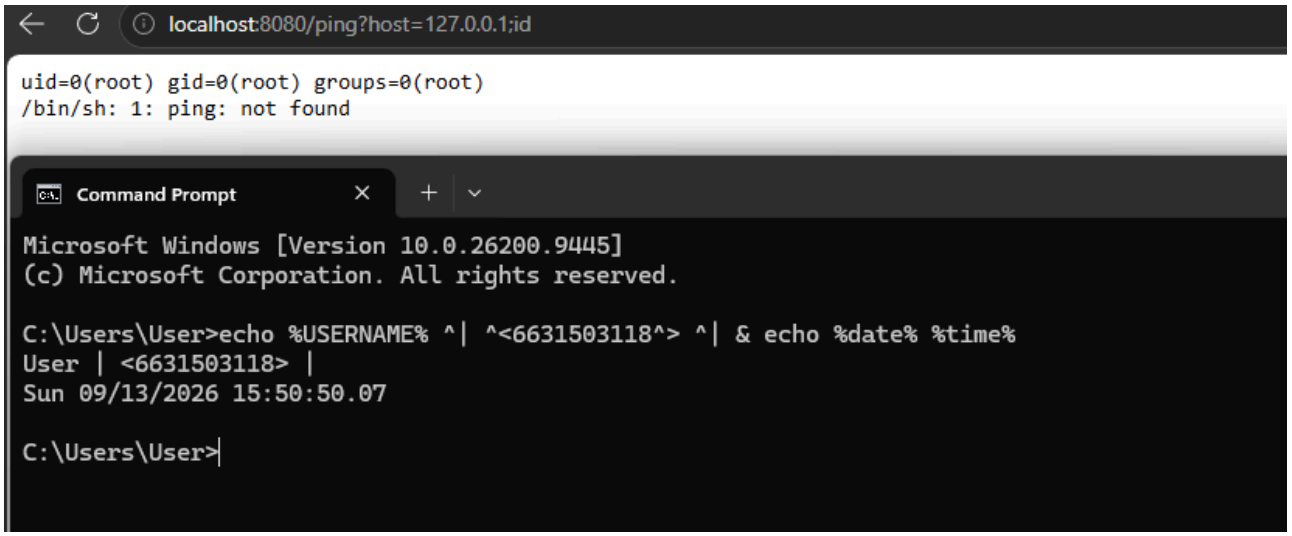
Task 2 — Credential dump via UNION SQLi (30 min) 🐍 Hit #2.

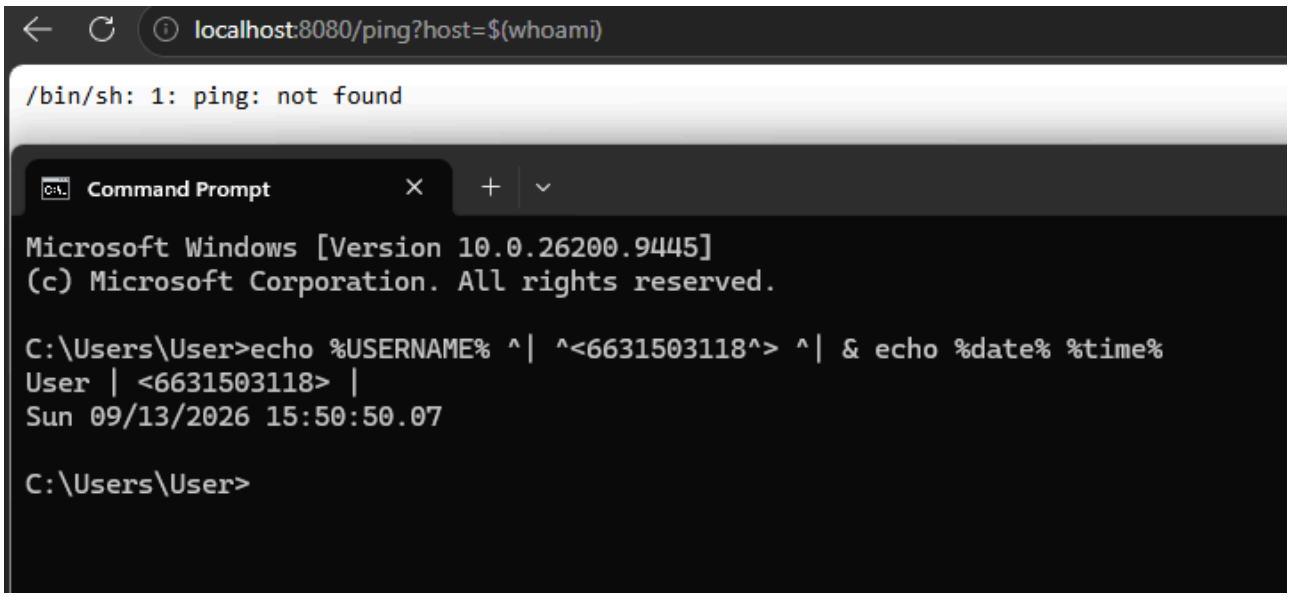
- *Goal:* exfiltrate every username **and password** from the `users` table.
- *Steps:* request `/search?q=' UNION SELECT username,password FROM users--`. Confirm `alice:alicepw` and `bob:bobpw` appear.
- *Deliverable:* payload + screenshot of dumped credentials + note on why column count must match.



Task 3 — OS command injection (30 min) 🐍 Hit #3.

- *Goal:* run an arbitrary command through `/ping`.
- *Steps:* request `/ping?host=127.0.0.1;id` then `/ping?host=$(whoami)` (URL-encode if needed). Capture the injected command's output.
- *Deliverable:* both payloads + screenshot of `id`/`whoami` output + explanation of the `shell=True` flaw (CWE-78).



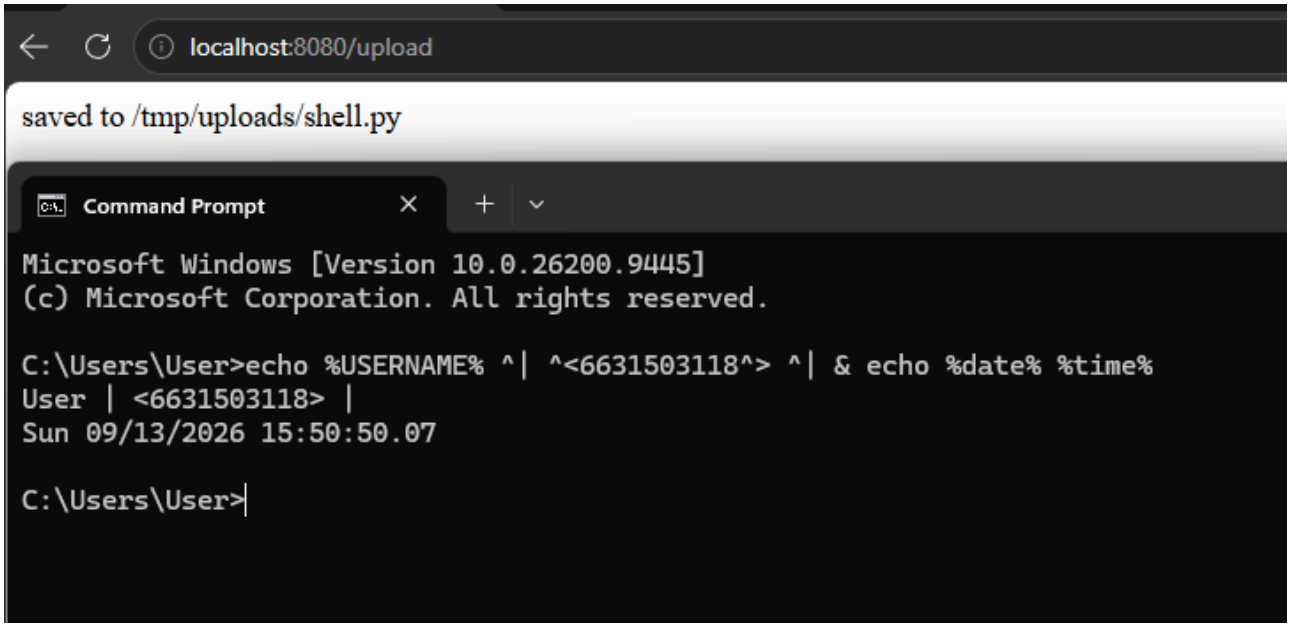


- shell=True means Python hands your string to /bin/sh, which treats ; and $() as shell metacharacters instead of a literal hostname (CWE-78). The fix is passing an argument array ["ping","-c","1",host] with no shell, so host can only ever be a single argument, never new commands.

Task 4 — Unrestricted upload (25 min) 🐍 Hit #4.

- *Goal:* show the upload accepts a dangerous file type with no checks (CWE-434).
- *Steps:* `GET /upload` (form), then upload a file named `shell.py`. Confirm `saved to /tmp/uploads/shell.py`. Discuss: if `UPLOAD_DIR` were web-served or executed, this is the RCE chain (here the dir is **not** served, so document the missing control rather than claiming auto-RCE).

- *Deliverable:* upload command/screenshot + 2–3 sentences on why extension allow-listing matters.



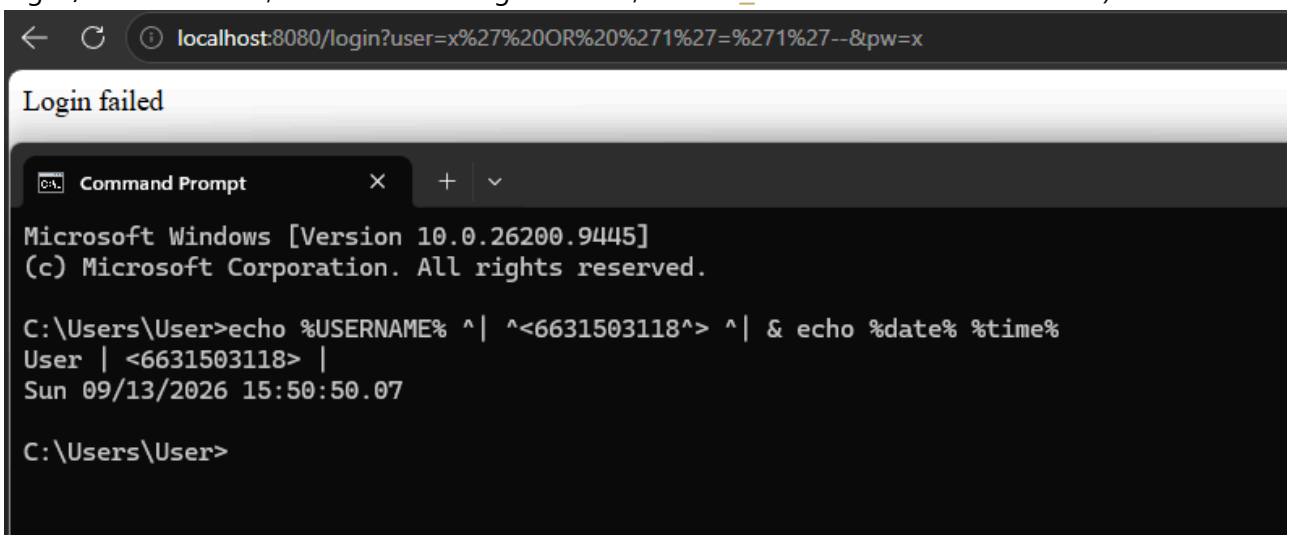
- The app calls f.save(dest) on any filename with no validation (CWE-434), so a .py/.php/.sh file lands on disk untouched. On its own that's not RCE — in this lab /tmp/uploads isn't web-served or executed, so nothing runs the file

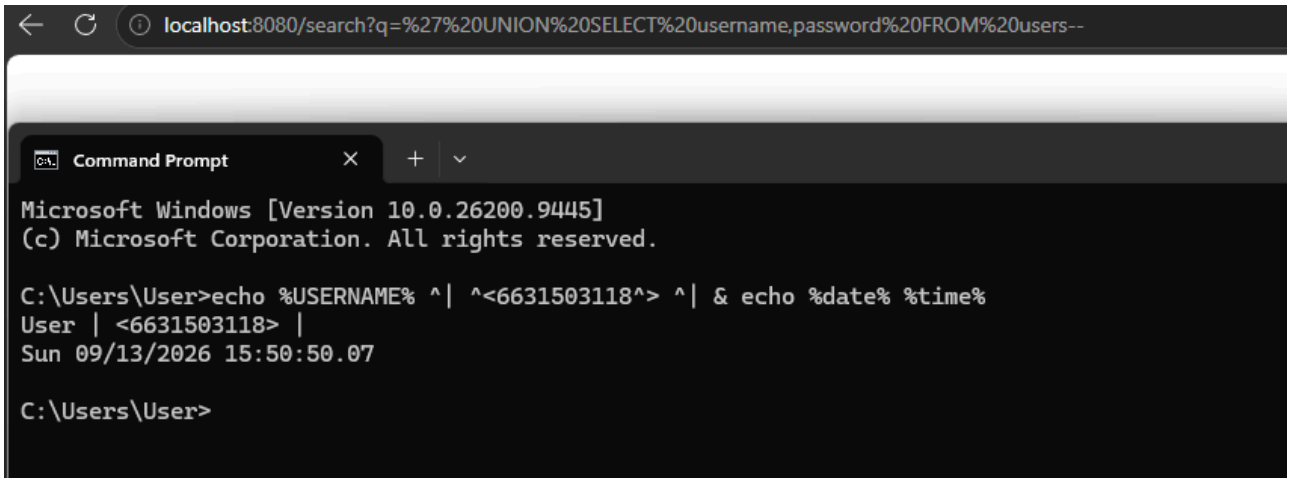
Task 5 — Defend / fix it (35 min) 🛡️ Boss defeated.

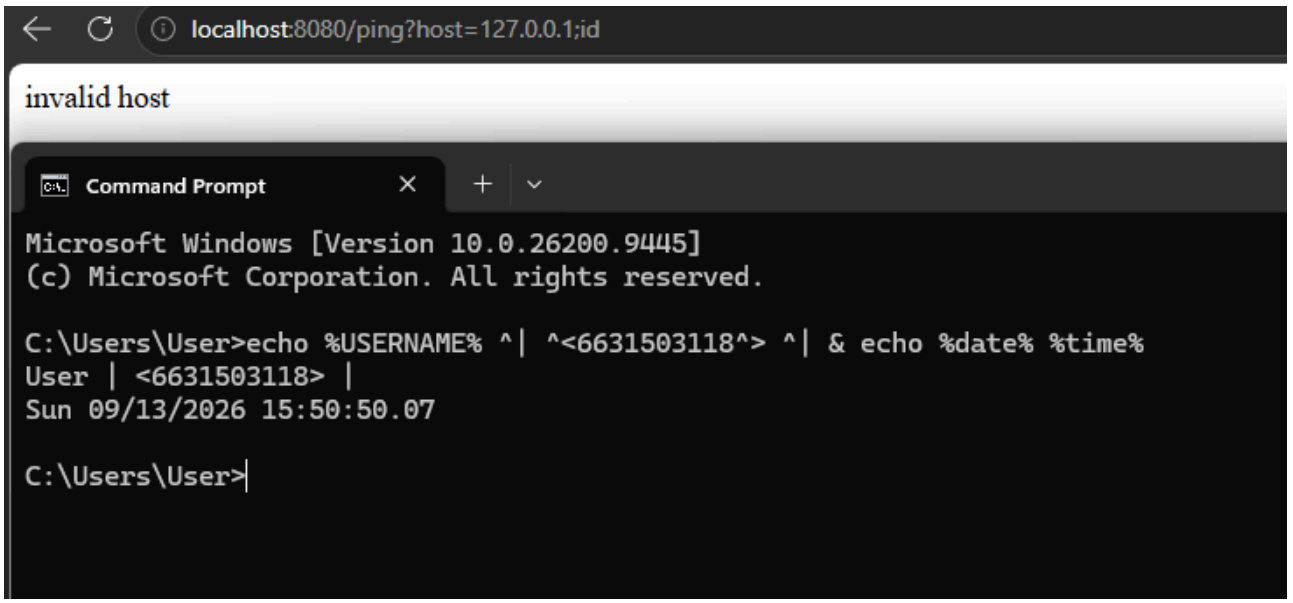
- *Goal:* prove `solution_app.py` blocks Tasks 1–4.
- *Steps:* stop the vulnerable container (`Ctrl-C`), then run the fixed app on the same compose env:

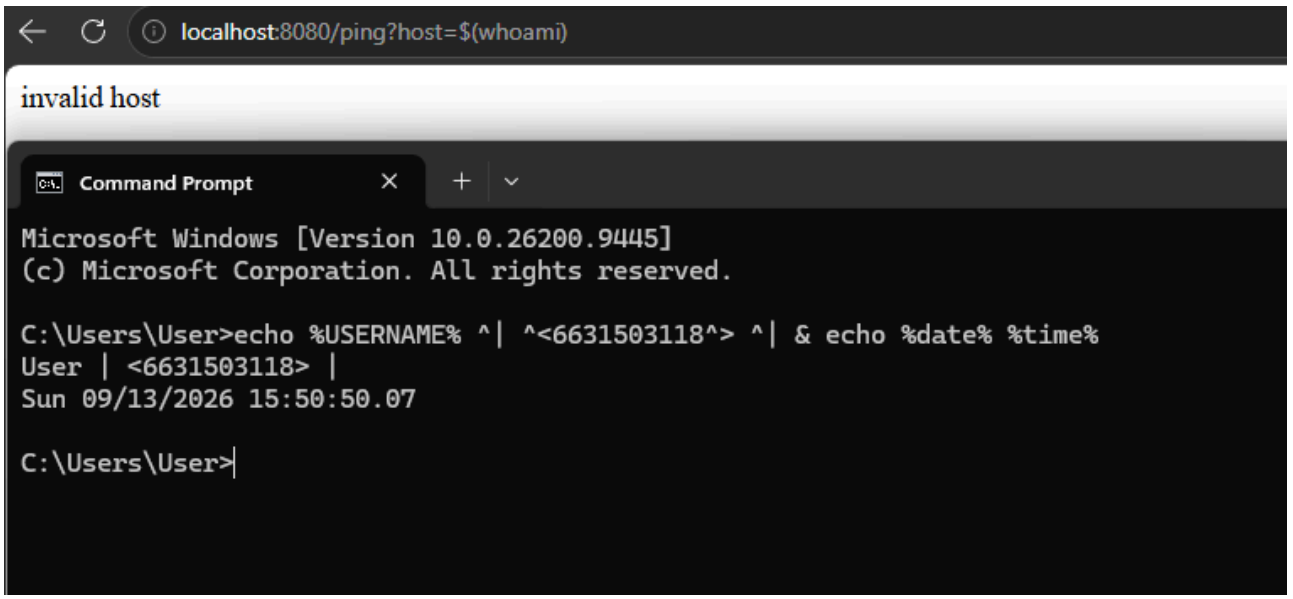
```
docker compose run --rm --service-ports injection-lab bash -c "pip install --no-cache-dir flask && python solution_app.py"
```

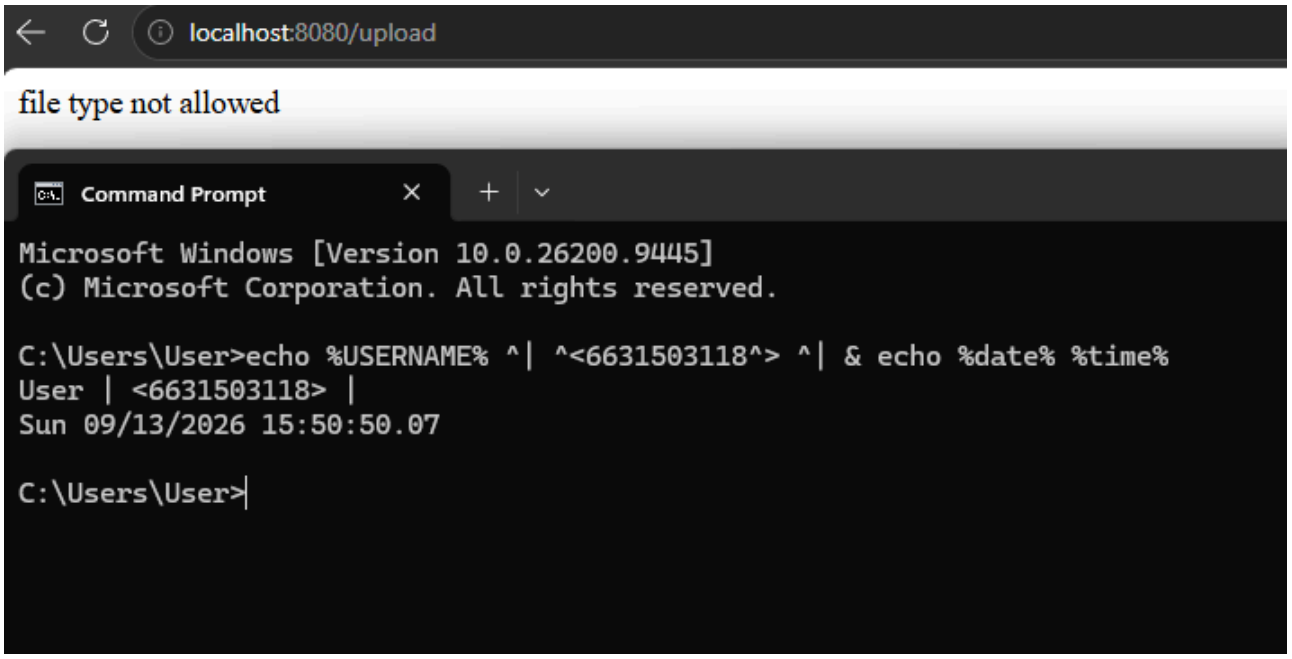
  Re-fire each payload from Tasks 1–4. Expected: `Login failed`, no credential dump, `invalid host` (400) on `127.0.0.1;id`, and `file type not allowed` for `shell.py`.
- *Deliverable:* screenshots of all four failures + name the fix line for each (parameterized query L52–55 login / L62–66 search, `shell=False`+regex L74–77, `secure_filename`+allow-list L86–93).

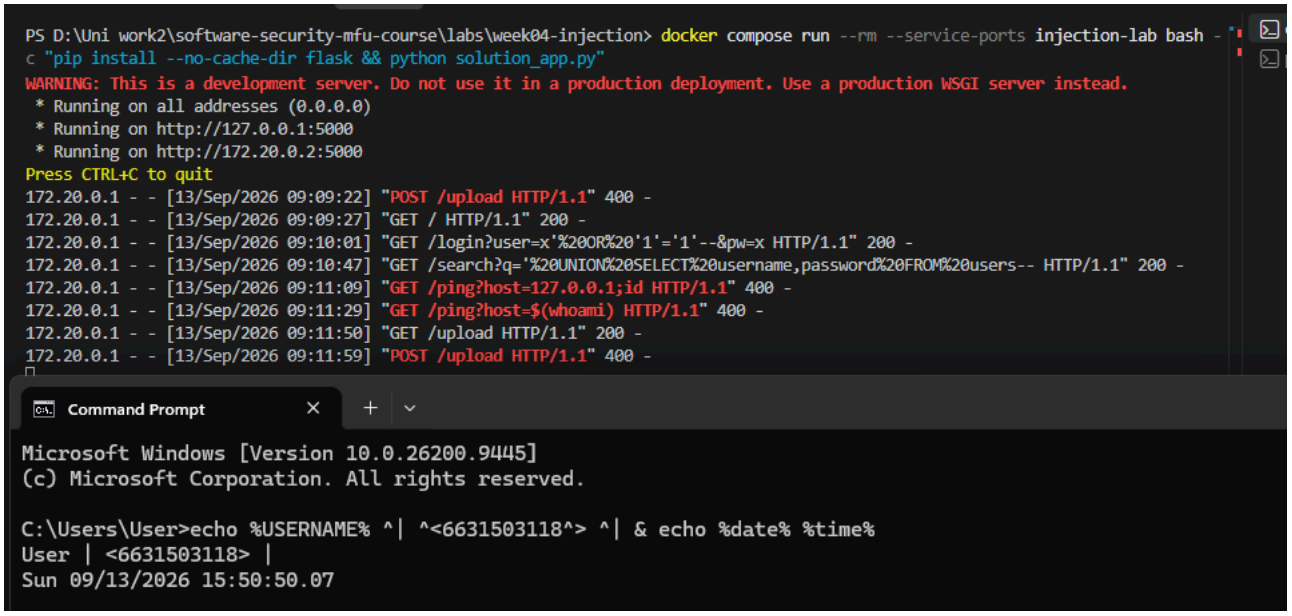



localhost:8080/search?q=%27%20UNION%20SELECT%20username,password%20FROM%20users--
Command Prompt
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.
C:\Users\User>echo %USERNAME% ^| ^<6631503118^> ^| & echo %date% %time%
User | <6631503118> |
Sun 09/13/2026 15:50:50.07
C:\Users\User>


localhost:8080/ping?host=127.0.0.1;id
invalid host
Command Prompt
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.
C:\Users\User>echo %USERNAME% ^| ^<6631503118^> ^| & echo %date% %time%
User | <6631503118> |
Sun 09/13/2026 15:50:50.07
C:\Users\User>


localhost:8080/ping?host=$(whoami)
invalid host
Command Prompt
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.
C:\Users\User>echo %USERNAME% ^| ^<6631503118^> ^| & echo %date% %time%
User | <6631503118> |
Sun 09/13/2026 15:50:50.07
C:\Users\User>





Part 4 — Reflection

1. **CWE/OWASP mapping:** map each of your four exploits to its CWE (89/78/434) and to OWASP 2025 **A05 Injection**.

Task	Exploit	CWE	OWASP 2025
1	Auth bypass (`OR '1'='1'--`)	**CWE-89** SQL Injection	A05 Injection
2	UNION credential dump	**CWE-89** SQL Injection	A05 Injection
3	`/ping?host=;id`	**CWE-78** OS Command Injection	A05 Injection
4	`shell.py` upload	**CWE-434** Unrestricted Upload of Dangerous File Type	A05 Injection

All four are A05 Injection because untrusted input crosses into an interpreter (SQL engine, OS shell, filesystem) that treats part of the input as *instructions* rather than *data*.

2. **Real breach:** the **2017 Equifax breach** exposed ~147M people after attackers exploited a known input-handling flaw (Apache Struts CVE-2017-5638). In 3–4 sentences, connect that failure to the lessons in this lab (untrusted input reaching a powerful interpreter; the cost of an unpatched/unvalidated input path).

   > *Answer:* The Struts flaw let attackers hide OS commands inside the HTTP `Content-Type` header, which Struts evaluated as an OGNL expression — so untrusted input reached a powerful interpreter and executed, mechanically identical to Task 3 where `host` reaches `/bin/sh` via `shell=True`. A patch had existed for months, so the ~147M-record exposure came not from an unknown flaw but from an unpatched, unvalidated input path left open. The lesson maps directly onto this lab: every request value must be treated as hostile, and the gap between "a fix exists" and "the fix is deployed" is where real breaches happen.

3. **Best mitigation:** of parameterized queries, allow-list validation, least privilege, and avoiding `shell=True`, which single control would have prevented the most damage in this lab, and why?

   > *Answer:* **Parameterized queries.** Two of the four exploits (Tasks 1 and 2) are SQLi, and the UNION credential dump is the highest-impact outcome in the lab — total disclosure of every user's password. Parameterization separates code from data at the driver level, so no amount of quote/comment trickery can change the query's structure; it is more robust than allow-list validation (which fails on one overlooked case) and than least privilege (which only limits, not prevents). Avoiding `shell=True` is the equivalent must-do for Task 3, but it defends fewer and lower-impact exploits here.

Grading rubric (100)

Criterion	Points
Part 2 — Lecture questions (conceptual accuracy)	20
Part 3 — Exploitation + evidence (payloads + screenshots, Tasks 1–4)	40
Part 3 — Defense (Task 5: fixes proven, lines cited)	25
Part 4 — Reflection (CWE/OWASP mapping, breach, mitigation)	15
Total	**100**

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.

  > *TODO (yours): paste your stamped screenshots for Tasks 1–5 here.*

- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide:* `SUBMISSION.md` *in the repo root).*

  > *TODO (yours): recover your flag via* `/ping?host=;cat /flag.txt` *and paste it above.*

- **Explain in your own words** *(graded on your reasoning, not copied text):*

  1. What did you do, and **why did the vulnerability work**?

     > *Draft to rewrite in your own words:* I glued attacker-controlled input straight into a SQL query, an OS command, and a filesystem path. The vulnerability worked because the input was never kept separate from the command — characters like `'`, `--`, and `;` stopped being data (a name, a hostname) and became syntax the interpreter executed.

  2. **Why does your fix actually stop it** — and what could still break it?

     > *Draft to rewrite in your own words:* The fixes tell each interpreter "this part is only data" — a parameterized query binds values so quotes can't change structure, an argument array with `shell=False` means `host` can only be one argument, and an extension allow-list plus `secure_filename()` rejects dangerous uploads and path traversal. It could still break if a fix is applied to one sink but the app grows a new unparameterized query, or if the allow-list is later widened to a deny-list.

Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.

   > *Example AI answer (replace with your own run):*
   >
   > ```
   > host = request.args.get("host", "127.0.0.1")
   > if ";" not in host and "|" not in host:      # block command injection
   >     out = subprocess.run("ping -c 1 " + host, shell=True,
   > capture_output=True, text=True)
   > ```

2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).

   > *Critique:* The blocklist on line 2 (`if ";" not in host and "|" not in host`) is incomplete — it misses `$(...)`, backticks, `&`, `&&`, and newlines, so the Task 3 payload `$(whoami)` passes straight through. It also keeps `shell=True`, so the shell interpreter is still reachable. This is the classic deny-list-instead-of-allow-list mistake.

3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

   > *Correct version:*

```
import re
host = request.args.get("host", "127.0.0.1")
if not re.fullmatch(r"[A-Za-z0-9.\-]+", host):
    return "invalid host", 400
out = subprocess.run(["ping", "-c", "1", host], capture_output=True,
text=True)  # no shell
```

Why the AI's was insufficient: A deny-list only blocks metacharacters someone remembered, and shells offer many ways to chain commands, so an attacker just uses one the filter forgot. The real fix removes the shell entirely (argument array, no `shell=True`) so `host` can only ever be one argument, and adds a positive allow-list regex that rejects anything that isn't a plausible hostname before it is used.

Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

Draft to rewrite in your own words: The app builds its SQL query and its ping command by gluing user input directly into a string, then hands that string to the database or the OS shell to run. Because the input is never kept separate from the command, characters like `'`, `--`, or `;` stop being part of a name and become new instructions, letting an attacker rewrite the query or run their own commands. It is exploitable precisely because the code never tells the interpreter "this part is only data."

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*

Final prompt: "Here is a Flask endpoint vulnerable to OS command injection because it uses `subprocess.run` with `shell=True` and string concatenation. Rewrite it securely using an argument array (no shell) AND an allow-list regex that only permits hostname characters `[A-Za-z0-9.-]`, returning HTTP 400 for anything else. Do not use a blocklist. Show the full fixed function. [paste the `/ping` code]"

Verified result: After applying the returned fix, re-firing `/ping?host=127.0.0.1;id` and `/ping?host=$(whoami)` both return `invalid host` (400) with no `id`/`whoami` output — exploit fails.